modifiedPointer-First Agent Constitution: Architectural Audit and Refactoring Strategy for Stela Task Governance

1. Executive Summary: The Imperative for Deterministic Governance

The rapid evolution of repository-resident Artificial Intelligence (AI) has precipitated a fundamental crisis in modern software governance: the irreconcilable tension between the probabilistic, fluid nature of generative Large Language Models (LLMs) and the rigid, binary requirements of structural integrity in software engineering. In the specific context of the Stela ecosystem, this tension has ceased to be a theoretical abstraction and has manifested as a critical operational bottleneck that threatens the long-term viability of the platform. The current architectural paradigm, while structurally sound in its original intent, suffers from a specific and debilitating implementation flaw identified as "Instruction Layer Bloat"—a reliance on dense, narrative-driven instructions within the TASK.md surface to enforce governance.[1] This reliance forces AI agents to interpret governance as "Soft Law"—guidelines to be followed if the context window permits—rather than "Hard Physics," or immutable constraints enforced by the operational environment.[2]

This report presents an exhaustive forensic audit of the Stela repository (Head eccc11128) and delineates the comprehensive "Pointer-First Agent Constitution" strategy. The analysis confirms that the system is currently in an "Un-serviceable" state due to three convergent failure modes: "Ghost Canon" (the persistence of outdated governance references such as TRUTH.md), "Context Hazards" (token saturation leading to signal degradation), and "Cognitive Friction" (hallucination loops regarding state pointers).[1] To resolve these systemic anomalies, this report mandates the immediate execution of **DP-OPS-0038: Pointer-First Agent Constitution & Task Governance Refactor**. This Dispatch Packet (DP) functions as the vehicle for a radical refactoring operation that shifts the locus of control from the prompt's context window to the repository's ops/ binary layer, transforming Stela from a probabilistic text generator into a deterministic state machine capable of high-entropy resilience.

2. Theoretical Framework: The Physics of Pointer-First Architecture

To fully grasp the necessity of the Pointer-First architecture, one must first dissect the "Crisis of Context" inherent in LLM operations within complex code repositories. The legacy model of "Prompt Engineering"—characterized by stuffing context windows with elaborate behavioral instructions, coding guidelines, and project history—is fundamentally flawed when applied to rigorous software engineering governance.[3]

2.1 The Mechanics of Signal Degradation and Context Drift

"Drift" in the Stela environment is defined as the gradual divergence between the system's intended state, as defined by its governance documentation (e.g., PoT.md), and its actual operational state.[1] In legacy architectures, agents are instantiated with "thick" context windows loaded with paragraphs of behavioral instructions. While intuitive to human operators, this approach suffers from a critical flaw identified in recent research on context engineering: as the context window expands with redundant text, the Signal-to-Noise Ratio (SNR) diminishes precipitously.[1]

In the specific case of Stela, this manifests as "Instruction Layer Bloat" within TASK.md. When an agent is presented with a 2,000-token prompt containing conflicting or repetitive instructions (e.g., "Read PoT.md" followed by a summary of PoT.md), the attention mechanism of the Transformer model typically dilutes the weight of the immutable "Hard Constraints" in favor of the more verbose narrative text.[1] This leads to "Ghost Canon"—a phenomenon where the agent follows the *flavor* of the instruction (e.g., "write clean code") rather than the *law* of the repository (e.g., "adhere to ops/lib/project/SCAFFOLD.md").

Forensic analysis of the repository dump [1] indicates that the current TASK.md creates a high-entropy environment where the agent must expend significant cognitive resources simply to parse the governance, leaving fewer resources for the actual payload execution. This violates the principle of "Extreme Ownership" [3], where the agent should focus on the health of the system rather than interpreting ambiguous rules.

2.2 Cognitive Friction: The Branch Name Paradox

A critical friction point identified in the cognitive logs of previous agent sessions is the "Branch Name Hesitation".[1] Agents operating under the current constitution frequently halt or hallucinate when asked to populate the "Required Work Branch" field in TASK.md. The conflict arises from a logical paradox embedded in the current workflow: the TASK.md template requires the agent to *declare* a work branch, yet the "Staffing Protocol" in PoT.md explicitly forbids the Contractor (AI) from creating branches or inventing repository structures.[1]

The agent is thus trapped in a logic loop: it cannot fill the field because the branch does not exist in the artifacts provided (the OPEN artifact only lists main [1]), but it cannot create the branch to satisfy the field because it lacks the authority. This results in "Placeholder Paralysis," where the agent fears that providing a "Not provided" value will trigger a linter

failure, leading to a stoppage of work.[1] The Pointer-First Constitution resolves this by moving the decision of branch naming from the probabilistic Agent layer to the deterministic Operator layer (via the ops/bin/open binary), ensuring that the agent merely observes state rather than attempting to create it.

2.3 The Filing Doctrine as a Semantic Map

A critical component of the Pointer-First system is the "Filing Doctrine," which maps the repository tree to specific cognitive functions. This is not merely file organization; it is a "Semantic Map" that allows the agent to navigate the system without hallucinating structure.

Directory	Semantic Definition	Cognitive Function	Operational Reality
ops/	**Run**	Execution: The "hands" of the system. Contains executable logic, binaries, and automation.	Agents execute tools here; they do not deliberate or edit unless explicitly tasked with platform engineering.
docs/	**Explain**	Memory: The "cortex" of the system. Contains manuals, rationale, and governance.	This is the source of truth (SSOT). Agents read here to understand *why*, but docs/ files effectively "do" nothing.
projects/	**Work**	Output: The "payload" where value is generated.	This is the only mutable surface for "Contractor" agents under normal DPs. It must be strictly isolated from platform logic.
storage/	**Trash**	Ephemeral: The "scratchpad." Explicitly excluded	Stores artifacts like handoff/ and dumps/. These

		from canon.	prove work was done but do not define the rules of the work.

Legacy tasks that embed execution logic (which belongs in ops/) or lengthy explanations (which belong in docs/) into the TASK.md file violate this doctrine.[1] A Pointer-First TASK.md respects this separation by containing only the coordination logic necessary to link a specific objective to the correct ops/ tools and docs/ references.

3. Forensic Audit of the Stela Repository (Head eccc11128)

A deep forensic audit of the provided repository dump (dump-full-main-eccc11128.txt) reveals significant discrepancies between the "Pointer-First" ideal and the current operational reality. These discrepancies serve as the primary friction points causing model hesitation and drift.

3.1 Ghost Canon: The Persistence of TRUTH.md

The most glaring violation of "Referential Integrity" is the persistence of TRUTH.md in the automation layer despite PoT.md being declared the "single constitutional document" in the preamble of PoT.md.[1]

Evidence of Split-Brain Governance:

In the file .github/workflows/sop_policing.yml, the automation logic for enforcing "Canon" changes includes a check for a file named TRUTH.md:

JavaScript

```
// "Canon" = anything that changes project meaning...
const canonChanged = names.some(n =>
  n === "TRUTH.md" ||
  n === "TASK.md" ||
  n.startsWith("docs/")...
);
```

However, the repository dump confirms that TRUTH.md does not exist in the file index; only PoT.md is present.[1] This indicates that TRUTH.md is an outdated rule or a "ghost" reference to

a file that has been renamed to PoT.md. This "Split-Brain" scenario forces the Integrator (Lead AI) or the automated gates to enforce a law that no longer exists.

Implication: This is the definition of "Ghost Canon"—outdated rules lingering in the automation layer that compete with active governance. When an agent reads PoT.md, it sees one set of laws. When it encounters the workflow output or instructions referencing TRUTH.md (as seen in the User Query's own "Logic Conflict Resolution" parameter), it sees another. This violation of the "Absolute Literalism" standard causes hesitation and drift.[1]

3.2 Instruction Layer Bloat in TASK.md

The current TASK.md file [1] is a primary source of context saturation. It contains redundant checks that violate the Pointer-First elimination rule:

1. **Redundant Context Loading:** Section 1 of TASK.md lists files that "Must read (always)" including PoT.md and SoP.md. It then requires the worker to echo "Loaded: PoT, SoP...". Section 3.2 repeats this exact requirement: "Worker must confirm loaded before acting."
2. **Forensic Impact:** This is "Instruction Layer Bloat." The requirement to load context should be enforced by the ops/bin/open binary, which generates the prompt.[1] If the prompt is generated correctly, the context is loaded. Demanding a verbal echo is a weak, probabilistic check compared to a deterministic manifest verification (tools/lint/context.sh). It consumes tokens and attention without adding verification value.[1]
3. **Jurisdictional Redundancy:** Section 3.3 repeats the "Worker Git Authority" rules (no branch creation, no pushes). These rules are already constitutional law in PoT.md Section 4.1.[1] Repeating them in every task file invites "Drift" if PoT.md is updated but TASK.md is not.

3.3 The Discrepancy in Read-in Order

The .github/copilot-instructions.md file defines a specific "Read-in Order" for Guest Contractors that further complicates the context landscape:

1. PoT.md (Policy of Truth + Filing Doctrine)
2. SoP.md (History Ledger)
3. TASK.md (Active Work Surface)
4. docs/INDEX.md (Navigation)

This conflicts with PoT.md Section 1.3 (Canon Surfaces), which explicitly defines the canonical wayfinding document as docs/MAP.md.[1] While docs/INDEX.md exists, PoT.md identifies docs/MAP.md as the official surface for continuity. Identifying INDEX.md as a "Source of Truth" in the Copilot instructions creates a secondary "Ghost Canon" hazard where rules might be inferred from an unauthorized navigation file.[1]

3.4 The Context Hazard in Manifests

A distinct vulnerability identified in the repository architecture is the "Context Hazard".[1] The Policy of Truth (PoT.md) explicitly defines a Context Hazard as "any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure".[1]

The global context (ops/lib/manifests/CONTEXT.md) is loaded at the start of every session via ops/bin/open. If the library of all possible tasks and skills were included here, two specific failures would occur:

1. **Token Saturation:** The context window would be flooded with irrelevant instructions (e.g., how to perform a database migration when the task is a CSS update), degrading the Agent's reasoning capacity for the specific task at hand.
2. **Recursive Drift:** If the context defines the agents, and the agents define the context, a self-referential loop is created that makes it impossible to isolate variables during debugging.

The Pointer-First constitution resolves this by strictly enforcing "Just-In-Time" (JIT) loading. Agents, tasks, and skills are never loaded globally. They are referenced via pointer only when the specific Dispatch Packet (DP) requires them.

4. Implementation Strategy: The Pointer-First Task Refactor

The "Refactoring Strategy for Stela Task Governance" [1] mandates a radical restructuring of the TASK.md file and the associated toolchain. This strategy moves from a "Manual" paradigm (where the agent is taught how to behave) to a "Dashboard" paradigm (where the agent tracks state and tools).

4.1 The New TASK.md Schema

The new schema is stripped of narrative instruction. It relies on the presence of specific pointers to enforce behavior.

Section 1: The Freshness Pointer (Replacing "Required Context Load")

Instead of listing files to read, Section 1 now points to the OPEN artifact generated by ops/bin/open.

- *Old:* "Must read PoT.md, SoP.md..."
- *New:* "Pointer: storage/handoff/OPEN-*.txt"
- *Rationale:* The agent is forced to look at the machine-generated state, eliminating the "Branch Creation Paradox" by grounding the agent in what exists rather than what is requested.[1]

Section 2: The Logic Pointers (Replacing "Behavioral Standards")

Instead of repeating "No contractions" or "No direct pushes," we use a ## Pointers section that references the binary gates.

- *Old:* "Refrain from using contractions."
- *New:* "Linguistic Precision: tools/lint/style.sh"
- *Rationale:* The agent does not need to be told "Don't use contractions"; it knows style.sh will kill the process if it does. This creates a "Pointer-First" entity.[1]

Section 4: The Mandatory Closing Block (Template-Based)

The closing block is formalized to ensure the "Varied Wording" requirement is met by design.

- *Constraint:* "Final Squash Stub" must differ significantly from "Primary Commit Header".
- *Rationale:* Explicitly stating the constraint right next to the field leverages the "Constraint Satisfaction" capabilities of the LLM. It transforms the requirement from a forgotten preamble instruction into an immediate, form-filling constraint.[1]

4.2 The Phased Execution Plan

To transition Stela to this new constitution, we must execute a multi-phase "Refactoring Operation" that cleanses the repository of "Ghost Canon" and "Instruction Bloat."

- **Phase 1: The Registry Cleanse:** The objective is to eliminate "Ghost Canon" (TRUTH.md references) and align PoT.md as the supreme authority. This involves searching and replacing TRUTH.md with PoT.md in .github/workflows/sop_policing.yml and .github/copilot-instructions.md.
- **Phase 2: The TASK.md Refactor:** This phase involves replacing the current TASK.md with the "Pointer-First" schema. It also requires updating ops/bin/open to populate the new "Session State" section automatically, resolving the branch creation paradox.
- **Phase 3: Toolchain Alignment:** This phase ensures the binaries referenced in the new TASK.md actually exist and enforce the rules. It includes updating tools/lint/style.sh to strictly enforce "Contraction Prohibition" and tools/verify.sh to fail if the Filing Doctrine is violated.
- **Phase 4: Certification:** The final phase involves running the new linters to verify the repository state.

5. Dispatch Packet: DP-OPS-0038

The following Dispatch Packet (DP) is constructed to execute the strategy outlined above. It serves as the primary artifact for this engagement, strictly adhering to the legacy TASK.md structure while introducing the new content.

DP-OPS-0038: Pointer-First Agent Constitution & Task Governance Refactor

3.1 Freshness Gate (Must Pass Before Work)

Base Branch: main

Required Work Branch: work/constitution-refactor-0038

Base HEAD: eccc11128

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD
- git status --porcelain

Worker must already be on the Required Work Branch listed above (no branch creation or switching). STOP if any mismatch. STOP if the Required Work Branch is missing. STOP if told to create or switch branches.

3.2 Required Context Load (Read Before Doing Anything)

Worker must confirm loaded before acting:

- PoT, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - .github/workflows/sop_policing.yml
  - .github/copilot-instructions.md
  - ops/bin/open
  - tools/lint/style.sh
  - tools/verify.sh
  - docs/GOVERNANCE.md

(OPEN is for Integrator/DP Writer state refresh;

Worker does not read OPEN). DISPOSABLE ARTIFACTS (chat logs, etc.) MUST NOT BE REFERENCED OR INCLUDED.

3.3 Scope and Safety

- **Objective:** Transition Stela to a Pointer-First Agent Constitution by eliminating Ghost Canon, refactoring TASK.md to a deterministic dashboard, and hardening the ops toolchain to enforce the Filing Doctrine.
- **Non-Goals (optional, drift-killer):**
  - No changes to projects/.
  - No changes to docs/library/agents or docs/library/skills (unless strictly required for validation).

Target Files allowlist (hard gate)

- .github/workflows/sop_policing.yml
- .github/copilot-instructions.md
- docs/GOVERNANCE.md
- ops/bin/open
- tools/lint/style.sh
- tools/lint/dp.sh
- tools/verify.sh
- TASK.md
- SoP.md
  Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.
- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/ (Overridden for this DP only: see Target Files)
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
- Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **PoT.md wins**.
  If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**

    - STOP if any placeholder text remains in final outputs (square-bracket tokens, to-do markers, to-be-determined markers, or standalone ellipses).
- STOP if required DP inputs are missing (do not guess).
- STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

3.4 Execution Plan (A-E Canon)

3.4.1 State

- **Context:** The current repository governance relies on narrative-driven "Soft Law" instructions in TASK.md and contains "Ghost Canon" references to TRUTH.md (which does not exist). This causes high cognitive friction, "Branch Name Hesitation," and signal degradation for agents.
- **Drift:** .github/workflows/sop_policing.yml references TRUTH.md. .github/copilot-instructions.md prioritizes docs/INDEX.md over docs/MAP.md. TASK.md contains redundant procedural text ("Instruction Bloat").
- **Desired State:** A "Hard Physics" environment where governance is enforced by binaries. TASK.md is a sparse dashboard pointing to ops/ tools. TRUTH.md references are replaced by PoT.md.

3.4.2 Request

Numbered tasks (concrete, no vibes):

1. **Phase 1: Registry Cleanse.** In .github/workflows/sop_policing.yml, replace the literal string "TRUTH.md" with "PoT.md". In .github/copilot-instructions.md, remove TRUTH.md references if any, and update the "Read-in Order" to replace 4. docs/INDEX.md with 4. docs/MAP.md. In docs/GOVERNANCE.md, remove any prescriptive rules that duplicate PoT.md and replace them with pointers to PoT.md.
2. **Phase 2: TASK.md Refactor.** Replace the entire content of TASK.md with the "Pointer-First" schema defined in the "Refactoring Strategy" document (Section 5.1). Ensure it includes the "Session State" section, "Logic Pointers" section, and the "Mandatory Closing Block" with template constraints.
3. **Phase 3: Toolchain Alignment.** Update ops/bin/open to automatically populate the new TASK.md Section 1 ("Session State") with the active branch and HEAD from the git environment. Update tools/lint/style.sh to strictly enforce "Contraction Prohibition" on all markdown files (fail on "don't", "can't", etc.). Update tools/verify.sh to fail if docs/ contains binaries or ops/ contains loose markdown (enforcing Filing Doctrine).
4. **Phase 4: Certification.** Update tools/lint/dp.sh to validate the new TASK.md heading structure (decimal headings) and ensure it accepts the new "Session State" block. Verify that running bash tools/lint/style.sh produces no errors on the new TASK.md.

Required Content (optional; use when doc/governance changes demand exact

text/format):

- **New TASK.md Section 1 Schema:**
  1. Session State (The Anchor)
  Pointer: storage/handoff/OPEN-*.txt (The generated session context)
  - Active Branch: {{BRANCH_NAME}} (Must match OPEN artifact)
  - Base HEAD: {{SHORT_HASH}} (Must match OPEN artifact)
    Context Manifest: ops/lib/manifests/CONTEXT.md (Checked by tools/lint/context.sh)
- **New TASK.md Section 4 Constraint:**
  Mandatory Closing Block
  Constraint: "Final Squash Stub" must differ significantly from "Primary Commit Header".

3.4.3 Changelog

- **Log (human-readable, 1–6 bullets):**
  - Replaced TRUTH.md with PoT.md in CI workflows and Copilot instructions (Ghost Canon remediation).
  - Refactored TASK.md to Pointer-First schema, eliminating instruction bloat.
  - Updated ops/bin/open to automate session state injection.
  - Hardened tools/lint/style.sh and tools/verify.sh to enforce PoT standards.
  - Updated docs/GOVERNANCE.md to point to PoT.md.

3.4.4 Patch / Diff

- **Format:** Unified diff (preferred) or anchored snippets.

3.4.5 Receipt (Required)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform (or repo-correct equivalent when scope differs)
- If dump refiners are used, include this fallback line in the DP:
  - Fallback:./ops/bin/dump --scope=platform --format=chatgpt --out=auto --bundle
- **Zero-Byte Check:** Verify dump is not empty (test -s <path>).
- Context lint (use repo-canonical command):
  - Prefer: bash tools/lint/context.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)
- Truth lint (when PoT or canon surfaces touched; use repo-canonical command):
  - Prefer: bash tools/lint/truth.sh
  - If not found and required: **STOP** and ask (do not invent a substitute)

Skill capture (required during normal DP processing)

- Workflow: run ops/lib/scripts/skill.sh harvest to auto-generate a draft with provenance and semantic collision checks.
- Review: inspect the draft in storage/handoff/ for heuristic accuracy (Hot Zone and High Churn). Refine Invocation guidance and Solution.
  Do not edit the Provenance block.
- Promote: run ops/lib/scripts/skill.sh promote <draft_path> or log "no new skill promoted" with a rationale in RESULTS.
- Legacy capture is forbidden. Do not manually write skill files; the provenance block is required.
- If a DP requires skill capture, the DP Target Files allowlist must include docs/library/SKILLS.md.
- Proof required in RESULTS:
  - Harvest-only: command invoked plus draft path and ls storage/handoff/ output.
- Promotion: command invoked plus grep -n "Promotion Packet:" docs/library/SKILLS.md and grep -n "docs/library/skills/S-LEARN-" docs/library/INDEX.md.

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0038-RESULTS.md
- storage/handoff/OPEN-PORCELAIN-work-constitution-refactor-0038-eccc111.txt
- storage/handoff/OPEN-work-constitution-refactor-0038-eccc111.txt
- storage/dumps/dump-full-YYYY-MM-DD-HHMMSS.txt
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

MANDATE: No receipt/proof bundle = **AUTOMATIC DISAPPROVAL**.

Closeout routine receipts not being produced CONSISTENTLY INCLUDING PORCELAIN, DUMP AND OPEN is a failure condition.

Partial receipts (e.g., missing OPEN artifacts when a session was started) are considered failures.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)

- Verification: (Output of proofs)

RESULTS file must include (minimum)

- Verification results (RUN/NOT RUN + reason + risk)
- Patch/diff proof bundle (status/diff list or direct diff)
- Receipt pointers (OPEN + PORCELAIN + DUMP outputs/paths)
- git diff --name-only output
- git diff --stat output
- NEXT: one single action

Storage artifact flow (local-only, untracked):

- OPEN and OPEN-PORCELAIN outputs land in storage/handoff/.
- Dump bundles land in storage/dumps/dump-full-YYYY-MM-DD-HHMMSS.txt (example; use actual dump filename).
- Results receipts land in storage/handoff/DP-OPS-0038-RESULTS.md.
- Operator stores DP drafts under storage/dp/intake/ and moves or copies to storage/dp/processed/ after completion (or when reusing a previously executed DP).
- These are retention paths only; they are not worker prerequisites unless the DP explicitly says so.

---

4. Closeout (Mandatory)

Worker closeout duties (end of session)

- Produce or update: storage/handoff/DP-OPS-0038-RESULTS.md.
- If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- Run ops/bin/prune (hygiene).
- Run ops/bin/llms (refresh context bundles).
- Ensure proof bundle checklist is satisfied (above).
  (No bundle = DISAPPROVE.)
- Fill out the Mandatory Closing Block (see below).
- Append a **TASK Work Log** entry (below) capturing:
  - what changed (high level)
  - gates/verification outcome (pass/fail or not run + reason)
  - blockers (if any)
  - **NEXT:** one single action

Mandatory Closing Block (Varied Wording Required)

Varied Wording provision: Each entry must use meaningfully distinct wording;

copy or minor tense changes are not acceptable. Entry 4 must differ from Entry 1.

1. Primary Commit Header (plaintext)
2. Pull Request Title (plaintext)
3. Pull Request Description (markdown)
4. Final Squash Stub (plaintext) (Must differ from #1)
5. Extended Technical Manifest (plaintext)
6. Review Conversation Starter (markdown)

Integrator/operator closeout duties

- Review diff vs DP scope/forbidden zones.
- Run repo-gates / required checks.
- Commit/push/open PR/merge (operator-controlled).

4.1 Thread Transition (Reset / Archive Rule)

Purpose: Prevent TASK from becoming a museum while preserving continuity.

When a DP is **complete** (merged) or **ended** (canceled/superseded):

1. Add a final Work Log entry that starts with: THREAD END: and includes:
   - outcome (merged/canceled/superseded)
   - pointer to the DP RESULTS file
   - NEXT: the single action (often “start new DP” or “await operator”)
2. Reset sections **2. Active Context** and **3. Current Dispatch Packet** to the next DP.
3. Keep the Work Log, but separate threads with a visible divider line:
   - ---
   - THREAD START: DP-OPS-0000 (use the actual next DP id)

(If the Work Log gets too long, it’s acceptable to keep only the most recent thread’s entries here and move older thread logs into the DP RESULTS file—only when explicitly directed.)

---

5. Work Log (Timestamped Continuity)

> **NOTICE:** This log is for the *active* DP only.
>
> Historical logs are archived in 'storage/archives/'. Do not rely on this log for long-term history.
>
> **Rule:** This is not a transcript. It is the durable breadcrumb trail.
>
> **Expectation:** Append a new timestamped entry on every DP closeout (no

exceptions).

Each entry ends with: NEXT: <one single action>.

- *2026-02-08 16:30* — THREAD START: DP-OPS-0038. Seed: Pointer-First Constitution Refactor (Ghost Canon elimination, TASK schema replacement, Toolchain hardening).
- *2026-02-08 16:35* — DP-OPS-0038: Defined work branch work/constitution-refactor-0038 and Base HEAD eccc11128. Prepared for dispatch. NEXT: Execute DP-OPS-0038.

---

6. Verification and Risk Analysis

The transition to a Pointer-First Constitution introduces significant structural changes. The following verification matrix establishes the criteria for success and the rollback procedures in the event of failure.

6.1 Verification Matrix

Target Component	Success Criteria	Verification Command	Failure Mode
TASK.md	New schema present; no "Instruction Bloat" narrative.	bash tools/lint/dp.sh --test	Linter failure due to unrecognized headings (if lint not updated).
sop_policing.yml	No references to TRUTH.md.	grep "TRUTH.md".github/workflows/*.yml (Must return 0)	CI failure on PR if ghost canon persists.
ops/bin/open	OPEN artifact includes branch name matching git.	./ops/bin/open --out=auto	Agent halts due to missing "Active Branch" in TASK.md.
tools/lint/style.sh	Fails on "don't" or "can't" in markdown.	echo "don't" > test.md; bash tools/lint/style.sh test.md	Non-zero exit code required.

6.2 Risk Mitigation

Risk: The Agent fails to parse the new TASK.md schema because it expects the old "Instruction" format. **Mitigation:** The new schema retains the high-level ## structure but changes the content to pointers. The "Logic Pointers" section explicitly links PoT.md as the interpretative key. The cognitive logs suggest that agents *prefer* explicit constraints over ambiguous text.[1]

Risk: ops/bin/open update breaks the prompt generation.

Mitigation: The DP requires a local verification of the OPEN artifact before the agent proceeds. If ops/bin/open fails, the agent hits a hard stop condition ("STOP if required artifacts are missing") rather than hallucinating a prompt.

7. Conclusion

The implementation of **DP-OPS-0038** transforms Stela from a repository governed by "Soft Law" to one governed by "Hard Physics." By eliminating Ghost Canon and Instruction Bloat, we maximize the Signal-to-Noise Ratio for future agents, ensuring that governance is not merely read but physically enforced by the environment. This shift is a prerequisite for higher-order autonomous capabilities and directly addresses the "Un-serviceable" state identified in the forensic audit. The resulting architecture aligns with the principles of extreme ownership and unyielding precision required for autonomous engineering.[3] The adoption of this Dispatch Packet represents the final necessary step to stabilize the Stela platform for payload operations.

Works cited

1. Refactoring Strategy for Stela Task Governance.pdf
2. Understanding AI Agent Reliability: Best Practices for Preventing Drift in Production Systems, accessed February 8, 2026, https://www.getmaxim.ai/articles/understanding-ai-agent-reliability-best-practices-for-preventing-drift-in-production-systems/
3. This gist provides structured prompting rules for optimizing Cursor AI interactions. It includes three key files to streamline AI behavior for different tasks. · GitHub, accessed February 8, 2026, https://gist.github.com/tmichett/a9c8bccaeb0a9c623f8c20744c7ae334